

Introduction to Computer Systems Design

CS3031 · IIT Madras · Week 2

Module 1: Boolean Algebra

Detailed Study Notes

Boolean Algebra: from a Problem to a Logic Expression

- ▶ Recap of Week 1 and the lesson plan
- ▶ Tic-tac-toe: turning a natural-language rule into logic
- ▶ Functional & timing specifications; combinational vs sequential logic
- ▶ Truth tables and why they grow as 2^n
- ▶ NOT / AND / OR, the axioms of Boolean algebra, duality
- ▶ The 11 laws & theorems (with proofs), De Morgan's laws, NAND/NOR equivalences
- ▶ Literals, implicants, minterms, maxterms, operator precedence
- ▶ Canonical forms and Sum-of-Products (SOP) — and why SOP works
- ▶ 32 numericals, 20 subjective Qs, 40 MCQs

Based on lecture slides by Ayon Chakraborty, Department of CSE, IIT Madras. Boxes marked *Beyond the Slides* add standard textbook context that is not on the slides. Handwritten annotations on the slides have been interpreted and typeset; where they were ambiguous, this is flagged.

Contents

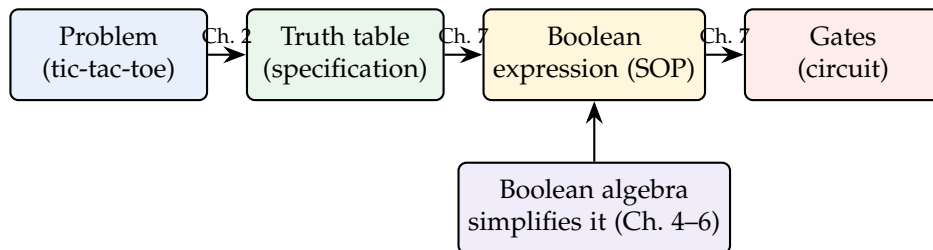
How to Use These Notes	4
I Module 1 — Boolean Algebra	6
1 Recap: Where Week 1 Left Us	7
1.1 The transformation hierarchy and the device hierarchy	7
1.2 Using voltages digitally	7
1.3 Digital logic $\rightarrow$ digital circuits	7
1.4 Lesson plan	8
2 From a Problem to a Logic Specification	9
2.1 The running example: tic-tac-toe	9
2.1.1 Why the “transformation” step matters	9
2.1.2 Why natural language is dangerous	9
2.2 Building complex systems: logic circuits and their two specifications	10
2.3 Combinational vs sequential logic	11
2.4 What exactly is a “function”?	11
3 Truth Tables	13
3.1 The truth table as a functional specification	13
3.1.1 Reading the two example rows	13
3.2 How big is a truth table?	13
3.3 The concise alternative: a Boolean expression	14
4 Boolean Algebra: Operations and Axioms	16
4.1 The three basic operations	16
4.2 Boolean algebra: the big picture	16
4.3 The axioms	17
4.3.1 Checking the axioms by truth table	17
4.4 Duality	18
4.4.1 Why duality works	18
5 Laws, Theorems and Proofs	20
5.1 The theorem table	20
5.2 Proving theorems from the axioms	21
5.3 Seeing the simplification theorems	22

5.4	Worked simplifications	23
6	De Morgan's Laws	24
6.1	Statement	24
6.1.1	Proof for two variables (perfect induction)	24
6.1.2	Proof from the axioms	24
6.1.3	Extending to n variables	24
6.2	Complementing a whole expression	25
6.3	De Morgan at the gate level: NAND and NOR	25
6.3.1	Bubble pushing	25
7	From Boolean Algebra to Logic Circuits: Sum of Products	27
7.1	Where we are in the hierarchy	27
7.2	The Sum-of-Products idea	27
7.3	Vocabulary	28
7.4	Operator precedence	28
7.5	Canonical (standard) forms	29
7.6	Two-level canonical form: Sum of Products (SOP)	29
7.6.1	The recipe	30
7.7	SOP: why does it work?	30
7.7.1	Proof that canonical SOP always equals F	30
7.8	From SOP to a circuit: two-level logic	31
II	Practice	32
8	Numerical Problems with Worked Solutions	33
8.1	Truth tables and counting	33
8.2	Evaluating expressions and precedence	34
8.3	Duals and complements	35
8.4	Simplification	35
8.5	Proofs	37
8.6	Canonical SOP	38
8.7	Gate equivalences	39
9	Subjective Questions with Answers	40
10	Multiple-Choice Questions	44
10.1	Questions	44
10.2	Answers and Explanations	50
A	Formula Sheet, Glossary and Revision	52

A.1	Formula and law sheet	52
A.2	Glossary	52
A.3	Revision checklist	53

How to Use These Notes

Week 2 finishes the *language* part of Module 1. Week 1 argued that computers should be digital and binary; Week 2 gives you the mathematics — Boolean algebra — for describing and manipulating binary functions, and the first systematic way (Sum of Products) of turning any truth table into a formula that can be built from gates.



Slides from the start of this deck repeat Week 1 material (transformation hierarchy, device hierarchy, digital voltages, Boole & Shannon). Chapter 1 recaps them briefly; see the Week 1 notes for the full treatment.

The coloured boxes

Intuition

The mental picture behind an idea.

Key Idea

The one sentence to remember.

Formula

A result with its derivation; sizes/counts in grey brackets like [rows] or [gates].

Common Pitfall

A common exam mistake.

Beyond the Slides

Standard background not on the slides (Mano; Harris & Harris).

Notation

Symbol	Name	Meaning
$\overline{A}$ (also A')	NOT / complement	1 iff $A = 0$
$A \cdot B$ or AB	AND / product	1 iff both are 1
$A + B$	OR / sum	1 iff at least one is 1
$A \oplus B$	XOR	1 iff exactly one is 1 (appears in Weeks 3–4)
n	number of input variables	
2^n	rows in a truth table	
m_i, M_i	minterm / maxterm number i	Week 3 uses these heavily

Precedence: NOT > AND > OR (Section 7.4).

Part I

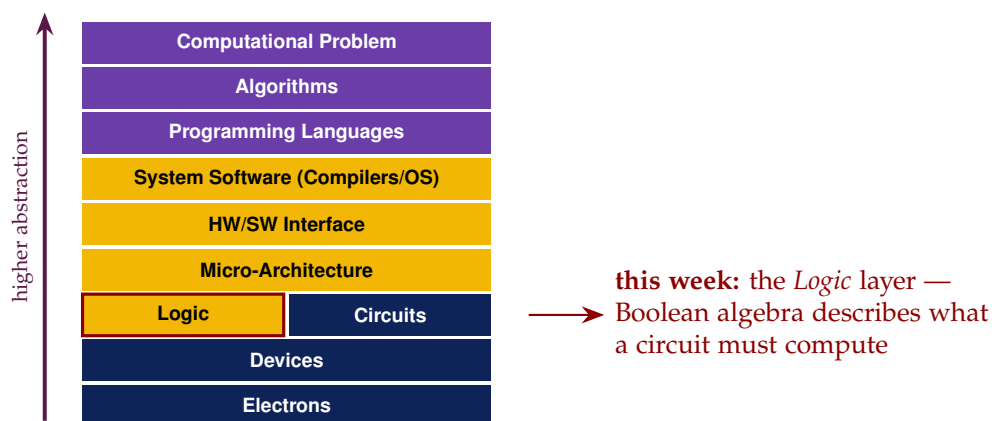
Module 1 — Boolean Algebra

Chapter 1

Recap: Where Week 1 Left Us

The first eight slides of the Week 2 deck repeat Week 1. They are summarised here because every idea in them is used again this week.

1.1 The transformation hierarchy and the device hierarchy



Physically, a phone is PCBs → ICs → modules → cells → gates → transistors, where information is ON/OFF **switching**. Gates are the level we now reason about.

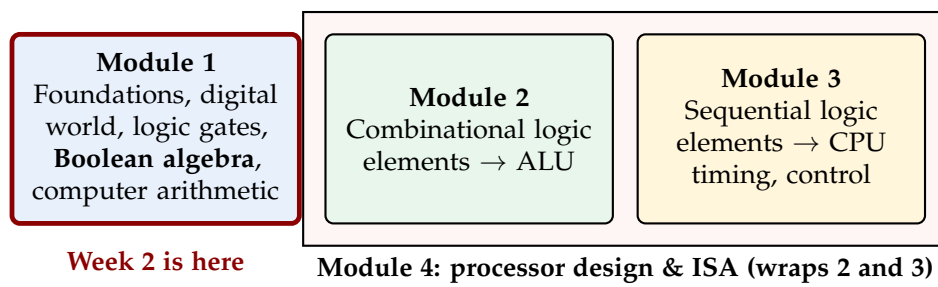
1.2 Using voltages digitally

- **Scheme #1:** a single threshold V_{TH} : $V < V_{TH} \Rightarrow 0$, $V \geq V_{TH} \Rightarrow 1$ — fragile near the threshold.
- **Scheme #2:** two thresholds: $V \leq V_L \Rightarrow 0$, $V \geq V_H \Rightarrow 1$, and a **forbidden zone** in between. (The slide prints " $V_L > V > V_H$ "; since $V_L < V_H$, read it as $V_L < V < V_H$.)
- Use the **same uniform representation** on every wire, so any output can drive any input.
- Binary digital system: two values, 0 and 1 (e.g. 0–0.5 V is "0", 2.5–3 V is "1"). Key advantage: **efficient circuits** — cheap, small, fast, low power.

1.3 Digital logic → digital circuits

- **1854, George Boole:** logic is mathematics; Boolean algebra is the mathematics of binary values.
- **1937, Claude Shannon:** relay/switch circuits obey Boolean algebra (series = AND, parallel = OR). Relays and vacuum tubes existed from 1906, but digital electronics waited thirty years for this theory.

1.4 Lesson plan



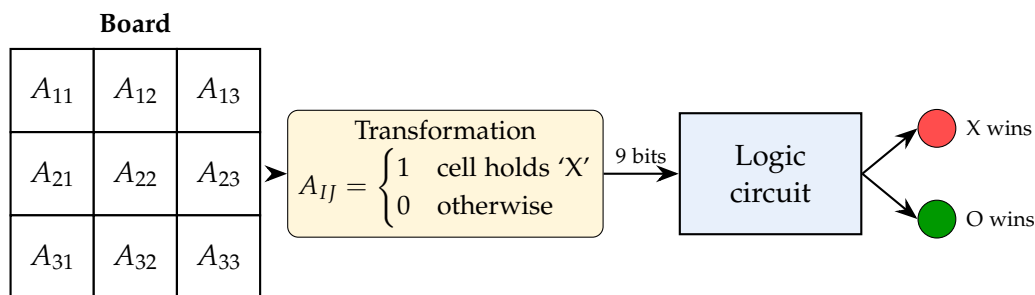
The slide draws Module 4 as a box *around* Modules 2 and 3: processor design uses the ALU (Module 2) and the timing/control (Module 3) together.

From a Problem to a Logic Specification

2.1 The running example: tic-tac-toe

From the slides:

Tic-Tac-Toe Game Assert whether the player with mark 'X' has won. Board cells $A_{IJ} = \text{'X'}$ or 'O'. **Transformation:** $A_{IJ} = 1$ if it contains 'X', $= 0$ otherwise. **Rule:** "If all three A_{IJ} s are 1 along any row, column or diagonal, the player 'X' wins." (Natural language — can be ambiguous!) Can we build a hardware circuit that checks the board configuration and declares the winner? (e.g., glows a RED LED if X wins, a GREEN LED if O wins.)



2.1.1 Why the "transformation" step matters

A circuit only understands 0s and 1s. So the first design step is always to decide *what each bit means* — an **encoding**. Here one bit per cell answers the single question "is there an X here?". This is exactly the transformation-hierarchy idea: a real-world problem is translated, layer by layer, into something hardware can process.

2.1.2 Why natural language is dangerous

The rule "if all three along any row, column or diagonal are 1, X wins" sounds precise, but ask:

- How many *diagonals*? Two (main and anti-diagonal) — "any diagonal" must mean both.
- What if both X and O have a line? That is impossible in a legal game but possible in an arbitrary 9-bit input. Should the circuit care?
- With $A_{IJ} = 0$ meaning "not X", an *empty* cell and an O look the same. So this encoding can detect "X wins" but can *not* detect "O wins". The slide's GREEN LED needs more information.

Key Idea

Hardware design starts by replacing a possibly-ambiguous English rule with an **exact specification**: a mapping from every possible input combination to an output. The rest of this week gives two exact languages for that: *truth tables* and *Boolean expressions*.

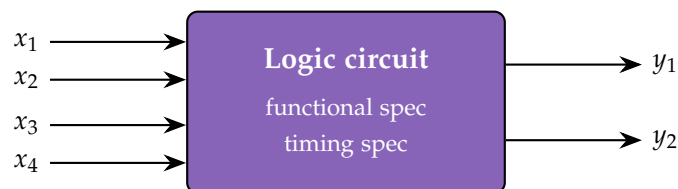
Beyond the Slides : encoding three states per cell

A cell can be empty, X, or O: three states, so one bit is not enough ($2^1 = 2 < 3$). With 2 bits per cell, say x_{IJ} ("has X") and o_{IJ} ("has O"), the board needs $9 \times 2 = 18$ input bits. "O wins" is then the same formula as "X wins" applied to the o_{IJ} bits. The pattern $x_{IJ} = o_{IJ} = 1$ is never produced by a legal board — a first example of input combinations we "don't care" about (Week 4).

2.2 Building complex systems: logic circuits and their two specifications

From the slides:

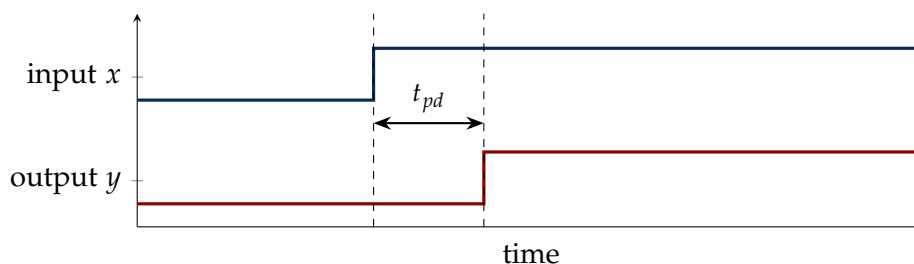
Building Complex Systems Build logic structures that, when suitably combined, evolve into a microarchitecture of a computing system. Input(s) $\rightarrow$ **Logic Circuit** (transistors implementing logic gates) $\rightarrow$ Output(s). **Functional specification:** relationship between input and output variables. **Timing specification:** the delay between inputs changing and outputs responding.



Functional: *what* the outputs are for each input combination (truth table / Boolean equations).

Timing: *how long* after an input changes the output becomes valid (propagation delay).

Weeks 2–4 are almost entirely about the **functional** specification. The slide's point is that a real circuit is not fully described until you also know its timing, because outputs never change instantly (Week 1's RC delay).

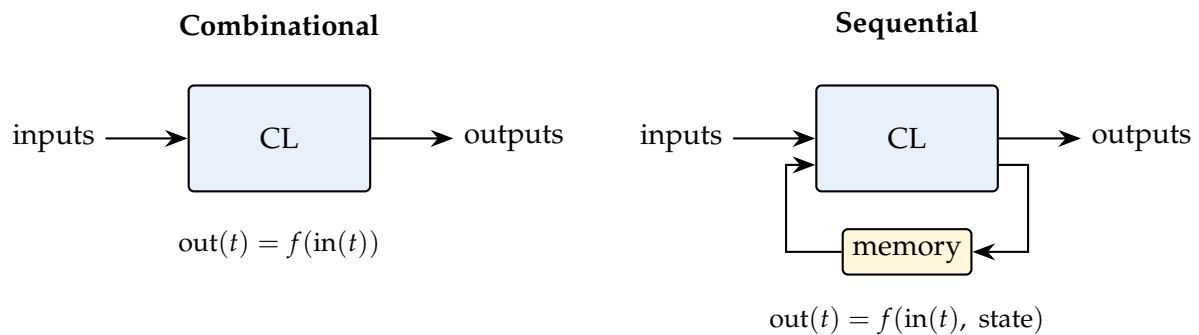


The output of a buffer follows its input only after the *propagation delay* t_{pd} . Timing specifications are studied properly with sequential logic (Module 3).

2.3 Combinational vs sequential logic

From the slides:

Building Complex Systems (2) **Combinational logic**: memoryless; outputs are strictly dependent on the combination of input values that are being applied to the circuit *right now*. **Later: sequential logic**: has memory; the structure stores history → can “store” data values; outputs are determined by previous (historical) and current values of inputs.



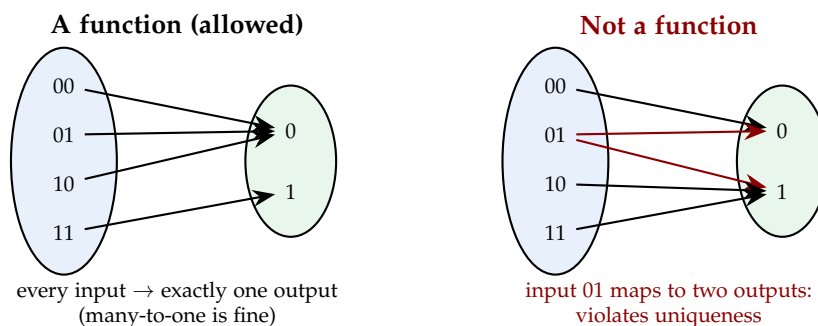
Intuition : calculator vs combination lock

A calculator's "+" key on two displayed numbers is combinational: the same two numbers always give the same sum. A combination lock is sequential: whether "7" opens it depends on which digits were entered *before*. Tic-tac-toe "has X won?" looks only at the current board, so it is combinational. "Whose turn is it?" depends on the history of moves, so it would need memory.

2.4 What exactly is a "function"?

From the slides:

Functional Specifications Functional specification of outputs in terms of inputs. What do we mean by "function"? **Uniqueness**: unique mapping from input values to output values. **Unambiguity**: the same input values produce the same output value every time. **No memory**: does not depend on the history of input values.



- **Uniqueness**: each input combination has exactly *one* output value. Many inputs may share an output (three inputs map to 0 above); what is forbidden is one input with two outputs.
- **Unambiguity**: repeat the same inputs tomorrow and you get the same output. No randomness.

- **No memory:** the output depends only on the present inputs — precisely the definition of combinational logic.

Key Idea

A combinational circuit *is* a (Boolean) function in the mathematical sense. That is why Boolean algebra — the algebra of functions on $\{0, 1\}$ — is the right tool for designing it.

Truth Tables

3.1 The truth table as a functional specification

From the slides:

Truth Table Functional specification: relationship between input and output variables. With $A_{IJ} = '1' \text{ or } '0'$, a table with columns $A_{11} A_{12} A_{13} A_{21} A_{22} A_{23} A_{31} A_{32} A_{33} | W$. Two example rows: 1 1 1 0 1 0 0 0 0 | 1 and 1 0 1 0 0 1 0 1 0 | 0, then "...". *Size of the table? Size grows exponentially with number of variables! Can we make it concise or easier to express/communicate?*

A **truth table** lists every possible combination of input values, one per row, and the output for each. Because it is exhaustive, it is an exact, unambiguous specification: it *is* the function.

3.1.1 Reading the two example rows

Row 1: $W = 1$

1	1	1
0	1	0
0	0	0

top row is all 1: X wins

Row 2: $W = 0$

1	0	1
0	0	1
0	1	0

no row, column or diagonal is all 1

The board is filled row by row: the 9 bits $A_{11}A_{12}A_{13} A_{21}A_{22}A_{23} A_{31}A_{32}A_{33}$ are laid out as three rows of three.

3.2 How big is a truth table?

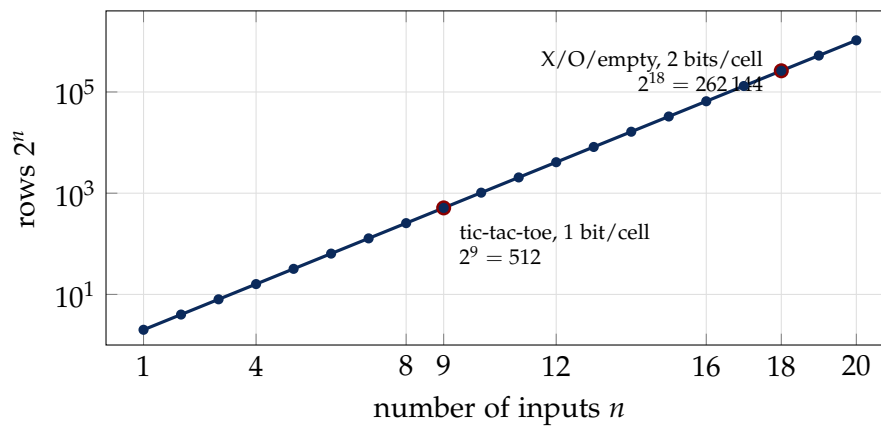
Formula : rows in a truth table

A function of n binary inputs has

$$R(n) = 2^n \quad [\text{rows}]$$

Derivation. Each row is a distinct assignment of 0/1 to n variables. By the multiplication principle (Week 1), there are $2 \times 2 \times \cdots \times 2 = 2^n$ such assignments.

For tic-tac-toe, $n = 9$: $R = 2^9 = 512$ rows, each with 9 input bits and 1 output bit. Brute-force counting shows that in 282 of those 512 rows at least one line is all 1s ($W = 1$).



Intuition : every extra input doubles the table

Adding one input means every existing row splits into two (new input = 0 and = 1). A 32-input function (a comparator for two 16-bit numbers) would need about 4.3 billion rows. Truth tables are exact, but hopeless beyond a dozen or so inputs. This is the slide's closing question: *can we make it concise?*

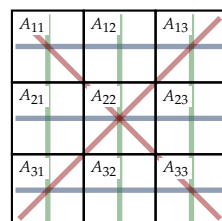
Formula : how many different functions of n inputs?

$$F(n) = 2^{2^n} \quad [\text{distinct Boolean functions}]$$

Derivation. A function is fixed by choosing the output column: one bit for each of the 2^n rows. Each bit has 2 choices, so there are $2^{(2^n)}$ possible output columns. For $n = 1$: 4 functions (0, 1, A , $\overline{A}$). For $n = 2$: 16 (AND, OR, NAND, NOR, XOR, XNOR, ...). For $n = 9$: 2^{512} — far more than atoms in the universe ($\sim 10^{80} \approx 2^{266}$).

3.3 The concise alternative: a Boolean expression

Let us translate the English rule directly. “All three in a line are 1” is an AND of the three cells. “Any line” is an OR over the 8 lines.



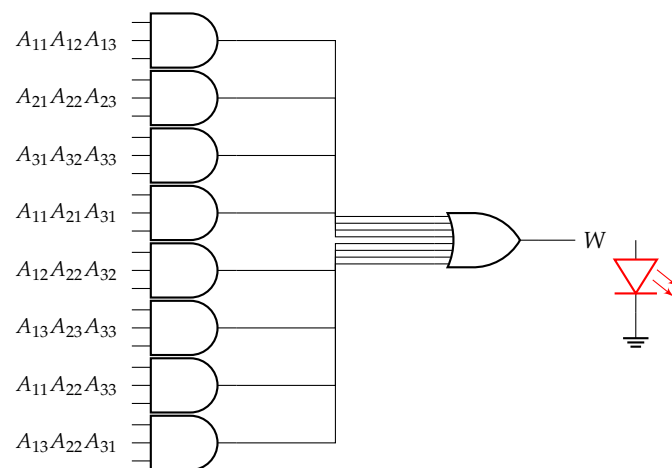
3 rows (blue) + 3 columns (green)
+ 2 diagonals (red)
= 8 winning lines

Formula : X-wins function

$$\begin{aligned} W = & A_{11}A_{12}A_{13} + A_{21}A_{22}A_{23} + A_{31}A_{32}A_{33} && (\text{rows}) \\ & + A_{11}A_{21}A_{31} + A_{12}A_{22}A_{32} + A_{13}A_{23}A_{33} && (\text{columns}) \\ & + A_{11}A_{22}A_{33} + A_{13}A_{22}A_{31} && (\text{diagonals}) \end{aligned}$$

8 product terms of 3 literals: 24 literals in total, versus a 512-row table.

This expression is already in **sum-of-products** form (Chapter 7): an OR of ANDs. It maps straight onto gates:



Each 3-input AND detects one line; the 8-input OR fires if any line is complete. The output drives the RED LED.

Key Idea

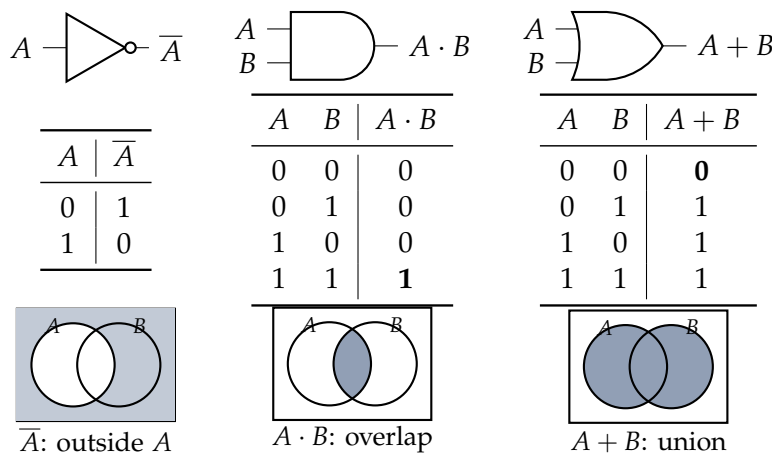
Truth table and Boolean expression are two descriptions of the same function. The table is exhaustive and easy to check; the expression is compact and maps onto gates. Boolean algebra, next, is the set of rules for manipulating expressions without changing the function they describe.

Boolean Algebra: Operations and Axioms

4.1 The three basic operations

From the slides:

Simple Equations: NOT / AND / OR $\bar{A}$ (reads “not A”) is 1 iff A is 0. $A \cdot B$ (reads “A and B”) is 1 iff A and B are both 1. $A + B$ (reads “A or B”) is 1 iff either A or B is 1. (Handwritten: the truth tables, “NOT: complement”, “AND”, “OR”.)



Intuition : three ways to see the same thing

- **Logic:** AND = “both true”, OR = “at least one true”, NOT = “the opposite”.
- **Arithmetic:** AND is ordinary multiplication on $\{0,1\}$ ($1 \cdot 1 = 1$, anything $\cdot 0 = 0$). OR is addition *capped at 1* ($1 + 1 = 1$). NOT is $1 - A$.
- **Sets (Venn diagrams):** variables are regions, AND is intersection, OR is union, NOT is the outside. Any identity that holds for regions holds in Boolean algebra, so Venn diagrams are a quick sanity check for the laws below.

Common Pitfall

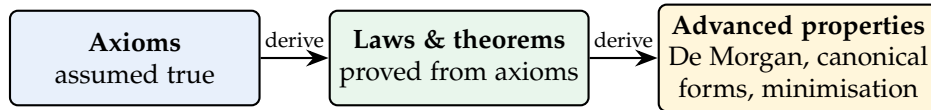
In Boolean algebra $1 + 1 = 1$, not 2 and not 10_2 . The “+” is OR, not binary addition. Binary addition (with carries) is built later *out of* these operations (Week 4: half/full adders).

4.2 Boolean algebra: the big picture

From the slides:

Boolean Algebra: Big Picture An algebra on 1s and 0s with AND, OR, NOT operations. **What you start with:** axioms — basic things about objects and operations you just assume to be true at the start. **What you derive first:** laws and theorems — allow you to manipulate

Boolean expressions and do some simplification. **What you derive later:** more “sophisticated” properties useful for manipulating digital designs represented as Boolean equations.



Like Euclid’s geometry: a few postulates, everything else proved.
Here the “everything else” is the toolkit for simplifying circuits.

4.3 The axioms

From the slides:

Boolean Algebra: Axioms Formal version with English version: (1) *B* contains at least two elements 0 and 1 such that $0 \neq 1$ — math formality. (2) Closure: $a, b \in B$: $a + b \in B$, $a \cdot b \in B$ — result of AND/OR stays in the set you start with. (3) Commutative: $a + b = b + a$, $a \cdot b = b \cdot a$ — order doesn’t matter. (4) Identities: $a + 0 = a$, $a \cdot 1 = a$ — identity elements give you back what you started with. (5) Distributive: $a + (b \cdot c) = (a + b) \cdot (a + c)$; $a \cdot (b + c) = a \cdot b + a \cdot c$ — $\cdot$ distributes over $+$ just like algebra, but $+$ distributes over $\cdot$ also. (6) Complement: $a + \bar{a} = 1$, $a \cdot \bar{a} = 0$ — there is a complement element; AND/ORing with it gives the identity element.

#	Name	Formal statement	What it means
A1	Two elements	$0, 1 \in B$, $0 \neq 1$	we work with (at least) two distinct values
A2	Closure	$a + b \in B$, $a \cdot b \in B$	AND/OR never produce anything but 0 or 1 — gates can feed gates
A3	Commutative	$a + b = b + a$; $a \cdot b = b \cdot a$	input order of a 2-input gate doesn’t matter
A4	Identity	$a + 0 = a$; $a \cdot 1 = a$	0 is “neutral” for OR, 1 is neutral for AND
A5	Distributive	$a \cdot (b + c) = ab + ac$; $a + bc = (a + b)(a + c)$	each operation distributes over the other
A6	Complement	$a + \bar{a} = 1$; $a \cdot \bar{a} = 0$	a signal and its inverse: exactly one is 1

Numbering note. The axiom slide numbers these 1–6. The proof slide later cites “Distributive (5)”, “Complement (6)”, “Identity (4)” — matching this list — but also “Identity (2)” for $1 + Y = 1$, which is actually *Theorem 2* of the next slide ($X + 1 = 1$), not an axiom. These notes label axioms A1–A6 and theorems T1–T12 to avoid the clash.

4.3.1 Checking the axioms by truth table

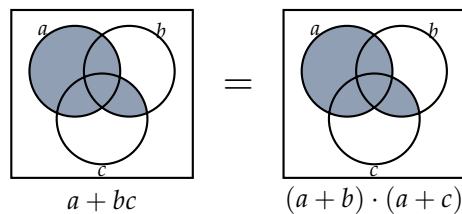
Since variables take only two values, any identity can be checked by trying every combination (*perfect induction*). The surprising one is the second distributive law, which fails for ordinary numbers ($2 + 3 \cdot 4 = 14$ but $(2 + 3)(2 + 4) = 30$). In Boolean algebra it holds:

a	b	c	bc	$a + bc$	$a + b$	$a + c$	$(a + b)(a + c)$
0	0	0	0	0	0	0	0
0	0	1	0	0	0	1	0
0	1	0	0	0	1	0	0
0	1	1	1	1	1	1	1
1	0	0	0	1	1	1	1
1	0	1	0	1	1	1	1
1	1	0	0	1	1	1	1
1	1	1	1	1	1	1	1

The two bold columns agree in all 8 rows, so $a + bc = (a + b)(a + c)$.

Intuition : why OR distributes over AND

If $a = 1$, both sides are 1 (OR with 1 is 1). If $a = 0$, the left side is bc and the right is $(0 + b)(0 + c) = bc$. Two cases, both agree. With ordinary numbers, “ $a +$ ” does not saturate, so the trick fails.

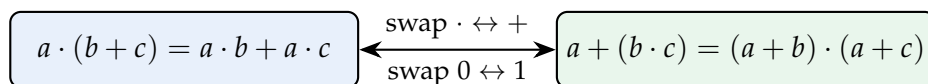


Shade $a + b$ and $a + c$ and intersect them: what survives is all of a plus the part of b that is also in c — the same region as $a + bc$.

4.4 Duality

From the slides:

Boolean Algebra: Axiom Duality All the axioms come in “dual” form. Anything true for an expression is also true for its dual, so any derivation that is true can be flipped into dual form and stays true. A **dual** is derived by replacing every AND with OR, every OR with AND, every constant 1 with 0, every constant 0 with 1 — but don’t change any of the literals or play with the complements! Example: $a \cdot (b + c) = (a \cdot b) + (a \cdot c) \rightarrow a + (b \cdot c) = (a + b) \cdot (a + c)$.



Variables and bars stay exactly where they are. Keep the original grouping: add brackets where needed, since AND binds tighter than OR.

4.4.1 Why duality works

Look at the axioms: each comes as a pair (i)/(ii), and each member of the pair is the dual of the other. Suppose you prove a theorem using a chain of axioms. Now write the dual of every line; each step is justified by the dual axiom, which is also an axiom. So the dual chain is also a valid proof, and the dual theorem is true. **Every theorem you prove gives you a second theorem for free.**

Beyond the Slides : duality at the gate level

There is a physical reading too. If you reinterpret every wire with the opposite convention (high voltage = 0, low = 1), an AND gate behaves as an OR gate and vice versa. The circuit hasn't changed; only the labels have. Duality is that relabelling written as algebra.

Common Pitfall

Dual $\neq$ complement. The dual of $a + b$ is $a \cdot b$; the complement of $a + b$ is $\bar{a} \cdot \bar{b}$ (De Morgan, Chapter 6). Duality swaps operators and constants but leaves variables alone; complementing also flips every variable.

Chapter 5

Laws, Theorems and Proofs

5.1 The theorem table

From the slides:

Boolean Algebra: Axiom Duality (theorem slides) Operations with 0 and 1: 1. $X + 0 = X$, 1D. $X \cdot 1 = X$; 2. $X + 1 = 1$, 2D. $X \cdot 0 = 0$. Idempotent: 3. $X + X = X$, 3D. $X \cdot X = X$. Involution: 4. $\overline{\overline{X}} = X$. Complementarity: 5. $X + \overline{X} = 1$, 5D. $X \cdot \overline{X} = 0$. Commutative: 6. $X + Y = Y + X$, 6D. $X \cdot Y = Y \cdot X$. Associative: 7. $(X + Y) + Z = X + (Y + Z) = X + Y + Z$, 7D. $(XY)Z = X(YZ) = XYZ$. Distributive: 8. $X(Y + Z) = XY + XZ$, 8D. $X + YZ = (X + Y)(X + Z)$. Simplification: 9. $XY + X\overline{Y} = X$, 9D. $(X + Y)(X + \overline{Y}) = X$; 10. $X + XY = X$, 10D. $X(X + Y) = X$; 11. $(X + \overline{Y})Y = XY$, 11D. $X\overline{Y} + Y = X + Y$.

#	Law	OR form / AND form (dual)	In words
T1	Identity	$X + 0 = X$ / $X \cdot 1 = X$	neutral elements
T2	Null (dominance)	$X + 1 = 1$ / $X \cdot 0 = 0$	1 dominates OR, 0 dominates AND
T3	Idempotent	$X + X = X$ / $X \cdot X = X$	repeating a signal adds nothing
T4	Involution	$\overline{\overline{X}} = X$ (self-dual)	two inverters = a wire
T5	Complementarity	$X + \overline{X} = 1$ / $X \cdot \overline{X} = 0$	a signal and its inverse
T6	Commutative	$X + Y = Y + X$ / $XY = YX$	order of inputs
T7	Associative	$(X + Y) + Z = X + (Y + Z)$ / $(XY)Z = X(YZ)$	grouping; allows many-input gates
T8	Distributive	$X(Y + Z) = XY + XZ$ / $X + YZ = (X + Y)(X + Z)$	factor / expand
T9	Combining (adjacency)	$XY + X\overline{Y} = X$ / $(X + Y)(X + \overline{Y}) = X$	a variable that appears both ways drops out
T10	Absorption	$X + XY = X$ / $X(X + Y) = X$	the bigger term swallows the smaller
T11	Redundant literal	$X\overline{Y} + Y = X + Y$ / $(X + \overline{Y})Y = XY$	a complemented copy is redundant
T12	De Morgan	$\overline{X + Y} = \overline{X}\overline{Y}$ / $\overline{XY} = \overline{X} + \overline{Y}$	Chapter 6

The slide pairs 11 and 11D the other way round (11: $(X + \overline{Y})Y = XY$; 11D: $X\overline{Y} + Y = X + Y$). They are duals of each other, so the pairing is the same.

Key Idea

T9, T10 and T11 are the “simplification theorems”. Every one of them removes a gate or a literal, which is why the slide marks them “useful for simplifying expressions”. In Week 3,

Karnaugh-map style minimisation is essentially T9 applied systematically.

5.2 Proving theorems from the axioms

From the slides:

Boolean Algebra: Proving EX: Prove $X \cdot Y + X \cdot \bar{Y} = X$: $X \cdot (Y + \bar{Y}) = X$ [Distributive (5)]; $X \cdot 1 = X$ [Complement (6)]; $X = X$ [Identity (4)]. EX2: Prove $X + X \cdot Y = X$: $X \cdot 1 + X \cdot Y = X$ [Identity (4)]; $X \cdot (1 + Y) = X$ [Distributive (5)]; $X \cdot 1 = X$ [Identity (2)]; $X = X$ [Identity (4)].

Proofs are chains of equalities, each step justified by one axiom (or an already-proved theorem). Here are all of the table's theorems, in an order where each proof uses only things proved before it.

T1 (Identity) — is axiom A4.

T5 (Complementarity) — is axiom A6.

T3: $X + X = X$

$X + X = (X + X) \cdot 1$	A4 (identity)
$= (X + X) \cdot (X + \bar{X})$	A6 (complement)
$= X + X \cdot \bar{X}$	A5 (+ over $\cdot$, read right to left)
$= X + 0$	A6
$= X$	A4

Dual (T3D, $X \cdot X = X$): write the dual of every line — $XX = XX + 0 = XX + X\bar{X} = X(X + \bar{X}) = X \cdot 1 = X$.

T2: $X + 1 = 1$

$X + 1 = (X + 1) \cdot 1$	A4
$= (X + 1)(X + \bar{X})$	A6
$= X + 1 \cdot \bar{X}$	A5
$= X + \bar{X}$	A4 (with A3: $1 \cdot \bar{X} = \bar{X} \cdot 1$)
$= 1$	A6

Dual: $X \cdot 0 = 0$.

T9: $XY + X\bar{Y} = X$ (slide EX)

$XY + X\bar{Y} = X(Y + \bar{Y})$	A5 (factor out X)
$= X \cdot 1$	A6
$= X$	A4

T10: $X + XY = X$ (slide EX2)

$$\begin{aligned}
 X + XY &= X \cdot 1 + XY && \text{A4} \\
 &= X(1 + Y) && \text{A5} \\
 &= X \cdot 1 && \text{T2 (with A3: } 1 + Y = Y + 1 = 1) \\
 &= X && \text{A4}
 \end{aligned}$$

Dual, T10D: $X(X + Y) = X$. Direct check: $X(X + Y) = XX + XY = X + XY = X$ by T3D then T10.

T11D: $X\bar{Y} + Y = X + Y$

$$\begin{aligned}
 X\bar{Y} + Y &= Y + X\bar{Y} && \text{A3} \\
 &= (Y + X)(Y + \bar{Y}) && \text{A5 (+ over } \cdot) \\
 &= (Y + X) \cdot 1 = X + Y && \text{A6, A4, A3}
 \end{aligned}$$

And T11: $(X + \bar{Y})Y = XY + \bar{Y}Y = XY + 0 = XY$ (A5, A6, A4).

T4: $\bar{\bar{X}} = X$ (involution)

First a lemma: *the complement is unique*. Suppose both y and z satisfy $X + y = 1$, $Xy = 0$ and $X + z = 1$, $Xz = 0$. Then

$$y = y \cdot 1 = y(X + z) = yX + yz = 0 + yz = Xz + yz = (X + y)z = 1 \cdot z = z.$$

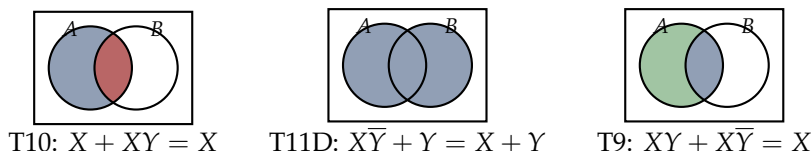
Now $\bar{X}$ satisfies $\bar{X} + X = 1$ and $\bar{X} \cdot X = 0$ (A6 with A3). So X is *a* complement of $\bar{X}$. By uniqueness, X is *the* complement of $\bar{X}$, i.e. $\bar{\bar{X}} = X$.

T7 (Associative)

Provable from the axioms, but the derivation is long. It is easiest to accept by **perfect induction**: check all $2^3 = 8$ assignments of X, Y, Z (Problem 26).

Common Pitfall

Don't "prove" a theorem by starting from it and simplifying both sides until you reach $X = X$ unless every step is reversible (an equality). The slide proofs are written in that style, and they are fine because each step replaces one side with an *equal* expression. Never divide or cancel: there is no subtraction or division in Boolean algebra, so $XY = XZ$ does *not* imply $Y = Z$ (take $X = 0$).

5.3 Seeing the simplification theorems

Read A as X and B as Y . T10: the overlap XY (red) already lies inside X , so adding it changes nothing. T11D: “ X but not Y ” plus “ Y ” is just “ X or Y ”. T9: the two halves of X (inside and outside Y) make up all of X .

Intuition : absorption in words

“It is raining, or it is raining and cold” says no more than “it is raining”. The second clause can only be true when the first already is. That is T10: the gate computing XY is useless in $X + XY$.

5.4 Worked simplifications

Worked Example : using T9

Simplify $F = ABC + ABC\bar{C}$.

$$F = AB(C + \bar{C}) = AB \cdot 1 = AB. \quad (\text{A5, A6, A4 — or T9 directly with } X = AB, Y = C)$$

Two 3-input ANDs and an OR shrink to one 2-input AND.

Worked Example : using T10 and T11

Simplify $G = \bar{A}B + \bar{A}BC + AB$.

$$G = \bar{A}B + AB$$

$$= B(\bar{A} + A) = B$$

$$\text{T10 with } X = \bar{A}B: \bar{A}B + \bar{A}BC = \bar{A}B$$

$$\text{T9 / A5, A6, A4}$$

Worked Example : using T11D

Simplify $H = \bar{X} + XY$. This is T11D with the roles renamed ($Y \cdot \bar{X} + \bar{X}$). Directly: $\bar{X} + XY = (\bar{X} + X)(\bar{X} + Y) = 1 \cdot (\bar{X} + Y) = \bar{X} + Y$ (A5, A6, A4). The X inside the product is redundant.

Beyond the Slides : the consensus theorem

One more simplification theorem you will meet in Week 3 and in textbooks:

$$XY + \bar{X}Z + YZ = XY + \bar{X}Z \quad (\text{dual: } (X + Y)(\bar{X} + Z)(Y + Z) = (X + Y)(\bar{X} + Z)).$$

Proof. $YZ = YZ(X + \bar{X}) = XYZ + \bar{X}YZ$. Then $XY + XYZ = XY$ and $\bar{X}Z + \bar{X}YZ = \bar{X}Z$ by absorption (T10). So the “consensus” term YZ is redundant. Intuition: whenever $YZ = 1$, either $X = 1$ (then $XY = 1$) or $X = 0$ (then $\bar{X}Z = 1$), so YZ never turns the output on by itself.

Chapter 6

De Morgan's Laws

6.1 Statement

From the slides:

DeMorgan's Laws 12. $\overline{X + Y + Z + \dots} = \overline{X} \cdot \overline{Y} \cdot \overline{Z} \dots$; 12D. $\overline{X \cdot Y \cdot Z \dots} = \overline{X} + \overline{Y} + \overline{Z} + \dots$. Think of this as a transformation: given $F = A + B + C$, applying (12) gives $\overline{F} = \overline{A + B + C} = \overline{A} \cdot \overline{B} \cdot \overline{C}$. "At least one of A, B, C is TRUE" $\leftrightarrow$ "It is not the case that A, B, C are all false."

Formula : De Morgan

$$\overline{X_1 + X_2 + \dots + X_n} = \overline{X_1} \cdot \overline{X_2} \dots \overline{X_n}$$

$$\overline{X_1 X_2 \dots X_n} = \overline{X_1} + \overline{X_2} + \dots + \overline{X_n}$$

In words: *break the bar, change the operator.*

6.1.1 Proof for two variables (perfect induction)

X	Y	$X + Y$	$\overline{X + Y}$	$\overline{X} \cdot \overline{Y}$	$\overline{X} \overline{Y}$	$\overline{X} + \overline{Y}$
0	0	0	1	1	1	1
0	1	1	0	0	1	1
1	0	1	0	0	1	1
1	1	1	0	0	0	0

Matching bold columns prove both laws.

6.1.2 Proof from the axioms

By uniqueness of the complement (Chapter 5), it is enough to show that $\overline{X} \overline{Y}$ behaves as the complement of $X + Y$, i.e. $(X + Y) + \overline{X} \overline{Y} = 1$ and $(X + Y) \cdot \overline{X} \overline{Y} = 0$:

$$\begin{aligned} (X + Y) + \overline{X} \overline{Y} &= (X + Y + \overline{X})(X + Y + \overline{Y}) && \text{A5} \\ &= (1 + Y)(X + 1) = 1 \cdot 1 = 1 && \text{A6, T2} \\ (X + Y) \overline{X} \overline{Y} &= X \overline{X} \overline{Y} + Y \overline{X} \overline{Y} && \text{A5} \\ &= 0 \cdot \overline{Y} + 0 \cdot \overline{X} = 0 && \text{A6, T2D} \end{aligned}$$

The second law is the dual.

6.1.3 Extending to n variables

Induction using associativity: $\overline{X + Y + Z} = \overline{(X + Y) + Z} = \overline{X + Y} \cdot \overline{Z} = \overline{X} \overline{Y} \overline{Z}$. Each step applies the 2-variable law.

Intuition : the slide's English reading

$A + B + C$ = “at least one of A, B, C is true”. Its negation is “none of them is true” = “A is false AND B is false AND C is false” = $\overline{A} \overline{B} \overline{C}$. Likewise, “not (all true)” = “at least one is false”. You already use De Morgan in everyday language; the law just writes it down.

6.2 Complementing a whole expression

To complement an expression: take the dual *and* complement every literal. Keep brackets.

Worked Example

$$F = A\overline{B} + \overline{C}D.$$

$$\overline{F} = \overline{A\overline{B} + \overline{C}D} = (\overline{A} + B)(C + \overline{D}).$$

Check one row: $A = 1, B = 0, C = 0, D = 0$: $F = 1 \cdot 1 + 1 \cdot 0 = 1$; $\overline{F} = (0 + 0)(0 + 1) = 0$. ✓

Common Pitfall

$\overline{AB} \neq \overline{A} \overline{B}$. Try $A = 1, B = 0$: $\overline{AB} = \overline{0} = 1$, but $\overline{A} \overline{B} = 0 \cdot 1 = 0$. A bar over a product is *not* a bar over each factor — the operator must change too.

6.3 De Morgan at the gate level: NAND and NOR**From the slides:**

DeMorgan's Laws (contd.) These are conversions between different types of logic functions. They can prove useful if you do not have every type of gate. $A = \overline{X} + \overline{Y} = \overline{X} \overline{Y}$: **NOR is equivalent to AND with inputs complemented**. $B = \overline{X} \overline{Y} = \overline{X} + \overline{Y}$: **NAND is equivalent to OR with inputs complemented**. (Each with a gate diagram and a truth table.)



NOR = AND with inverted (bubbled) inputs



NAND = OR with inverted (bubbled) inputs

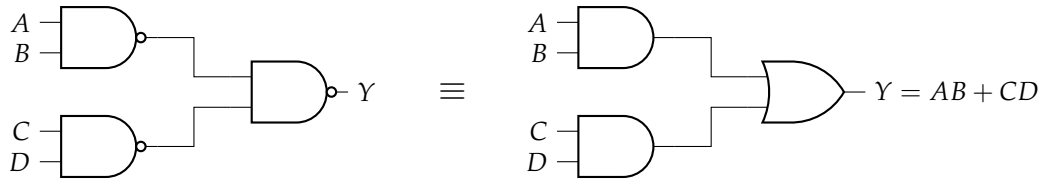
X	Y	$\overline{X}$	$\overline{Y}$	NOR = $\overline{X} \overline{Y}$	NAND = $\overline{X} + \overline{Y}$
0	0	1	1	1	1
0	1	1	0	0	1
1	0	0	1	0	1
1	1	0	0	0	0

6.3.1 Bubble pushing

A small circle (“bubble”) on a gate pin means inversion. De Morgan says you may:

1. push a bubble from the output of a gate to all its inputs (or back), and
2. at the same time swap AND $\leftrightarrow$ OR.

Two bubbles on the same wire cancel (involution, T4). This is how designers read and simplify circuits built from NAND and NOR gates, which are the cheapest gates in CMOS.



Derivation. $Y = \overline{\overline{AB} \cdot \overline{CD}} = \overline{\overline{AB}} + \overline{\overline{CD}} = AB + CD$ (De Morgan, then involution). The last NAND is an OR with bubbled inputs; those bubbles cancel the output bubbles of the first two NANDs. So a two-level NAND–NAND circuit computes a sum of products. Week 3 builds on exactly this (“universal gates”).

Key Idea

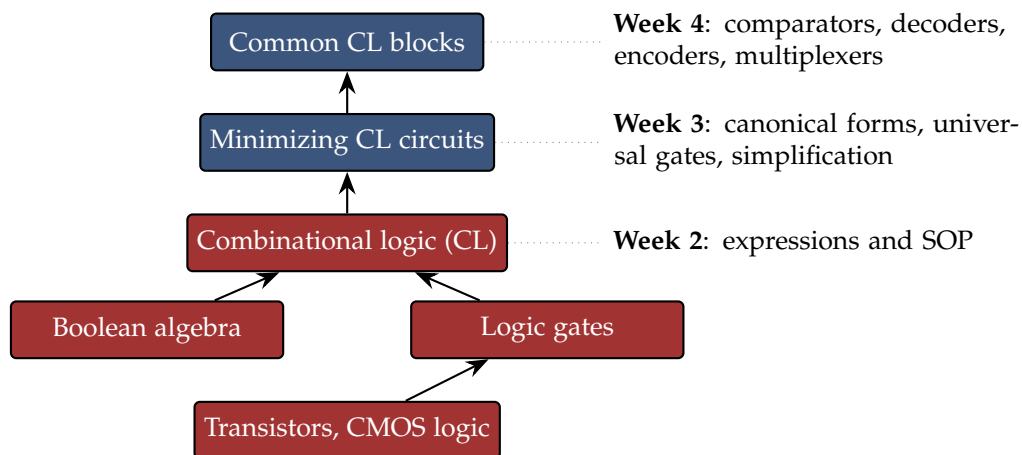
De Morgan lets you trade AND for OR at the cost of inversions. That is why a single gate type (NAND, or NOR) is enough to build any circuit — the topic of Week 3.

From Boolean Algebra to Logic Circuits: Sum of Products

7.1 Where we are in the hierarchy

From the slides:

Recall the Transformation Hierarchy Zooming into the Logic layer: Transistors, CMOS logic → {Boolean Algebra, Logic Gates} → Combinational Logic (CL) → Minimizing CL Circuits → Common CL Blocks.



7.2 The Sum-of-Products idea

From the slides:

Sum of Products Assume we have the truth table of a Boolean function. How do we express the function in terms of the inputs in a standard manner? Idea: **Sum of Products** form. Express the truth table as a two-level Boolean expression that contains all input variable combinations that result in a 1 output. If ANY of the combinations of input variables that results in a 1 is TRUE, then the output is 1. $F = \text{OR of all input variable combinations that result in a 1.}$

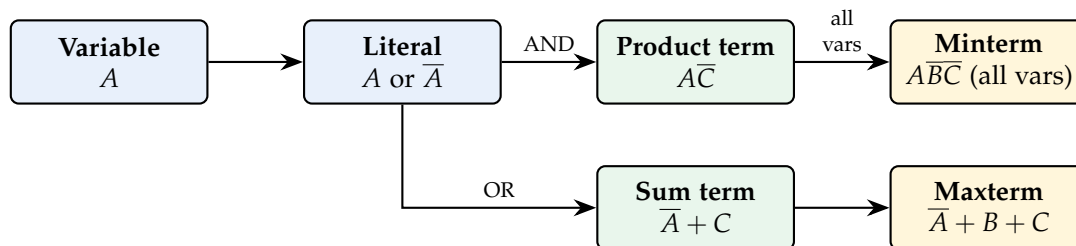
Intuition : a checklist of winning situations

Write down every situation in which the output should be 1. For each situation, build a detector that fires only in that exact situation (an AND). Then OR all the detectors: the output is 1 exactly when one of the listed situations is happening. You never need to think about when the output is 0 — it is 0 whenever no detector fires.

7.3 Vocabulary

From the slides:

Some Definitions **Complement:** variable with a bar over it ($\bar{A}, \bar{B}, \bar{C}$). **Literal:** variable or its complement ($A, \bar{A}, B, \bar{B}, C, \bar{C}$). **Implicant:** product (AND) of literals ($A \cdot B \cdot \bar{C}, \bar{A} \cdot C, B \cdot \bar{C}$). **Minterm:** product (AND) that includes *all* input variables ($A \cdot B \cdot \bar{C}, \bar{A} \cdot \bar{B} \cdot C, \bar{A} \cdot B \cdot \bar{C}$). **Maxterm:** sum (OR) that includes all input variables ($A + \bar{B} + \bar{C}, \bar{A} + B + \bar{C}, A + B + \bar{C}$).



Expression (inputs A, B, C)	Kind	Literals	Rows where it is 1
$\bar{A}$	literal / product of 1 literal	1	4 of 8
$A\bar{C}$	product term (implicant)	2	2 of 8
$A\bar{B}\bar{C}$	minterm	3	exactly 1
$\bar{A} + B + C$	maxterm	3	7 of 8 (0 in exactly 1 row)

Formula : rows covered by a product term

A product of k distinct literals, over n input variables, is 1 in

$$2^{n-k} \text{ [rows]}$$

Derivation. The product is 1 only if each of its k literals is 1, which fixes k variables. The remaining $n - k$ variables are free: 2^{n-k} combinations. A minterm has $k = n$, so it is 1 in exactly $2^0 = 1$ row. A maxterm, by duality, is 0 in exactly 1 row.

Beyond the Slides : “implicant” in textbooks

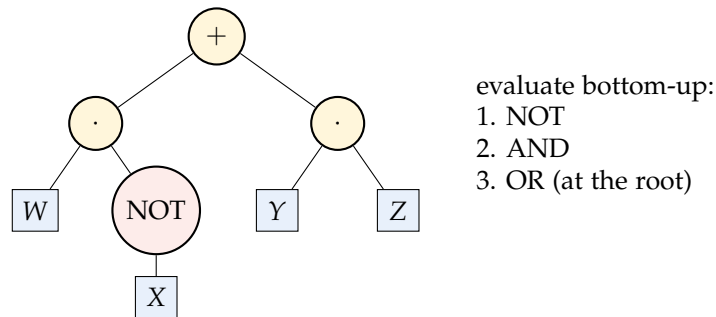
The slide uses *implicant* for any product of literals. In textbooks (Harris & Harris, Mano) an implicant is a product term of a particular function F that implies F : whenever the product is 1, F is 1. For example, for $F = A + BC$, $A\bar{B}$ is an implicant (whenever $A\bar{B} = 1$, $A = 1$ so $F = 1$), but $\bar{A}B$ is not (at $\bar{A}B\bar{C}$, $F = 0$). Week 3’s minimisation looks for the largest implicants (“prime implicants”).

7.4 Operator precedence

From the slides:

Operator Precedence What does $W \cdot \bar{X} + Y \cdot Z$ mean? Operator precedence rules: 1. NOT (highest priority); 2. AND; 3. OR (lowest).

$$W \cdot \bar{X} + Y \cdot Z = (W \cdot (\bar{X})) + (Y \cdot Z)$$

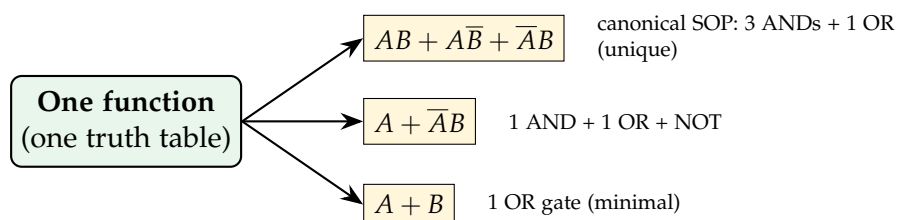


This mirrors ordinary algebra ($-$, then $\times$, then $+$) and it matches two-level circuits: inverters first, then an AND layer, then an OR layer. A bar drawn over a whole sub-expression acts like brackets: $\overline{A + B}$ means $\text{NOT}(A + B)$, which is *not* $\overline{A} + B$.

7.5 Canonical (standard) forms

From the slides:

Canonical (Standard) Forms Truth table is the unique signature of a Boolean function ... but it is an expensive representation (recall tic-tac-toe). A Boolean function can have many alternative Boolean expressions — i.e., many alternative expressions (and gate realizations) may have the same truth table (and function). **Canonical form:** standard form for a Boolean expression; provides a unique algebraic signature. If they all say the same thing, why do we care? Different Boolean expressions lead to different gate realizations.



All three expressions have the truth table 0, 1, 1, 1 (the OR function), but they cost very different amounts of hardware.

Key Idea

There are infinitely many expressions for any function, so comparing two expressions directly is hard. A **canonical form** is a rule that produces exactly one expression per function — so two expressions are equal iff their canonical forms are identical. Canonical SOP (below) is such a rule; it is *not* usually the cheapest circuit, which is why minimisation (Week 3) comes next.

7.6 Two-level canonical form: Sum of Products (SOP)

From the slides:

Two Level Canonical Form Sum of Products Form (SOP), also known as **disjunctive normal form** or **minterm expansion**. For the truth table with $F = 1$ in rows 011, 100, 101, 110, 111: $F = \overline{A}BC + A\overline{B}C + A\overline{B}\overline{C} + AB\overline{C} + ABC$. Each row in a truth table has a minterm; a minterm is a product (AND) of literals; each minterm is TRUE for that row (and only that row). All Boolean equations can be written in SOP form. Find all the input combinations (minterms) for

which the output of the function is TRUE.

7.6.1 The recipe

1. Find every row where $F = 1$.
2. For each such row, write the minterm: a variable that is **1** appears **plain**, a variable that is **0** appears **complemented**.
3. OR all these minterms together.

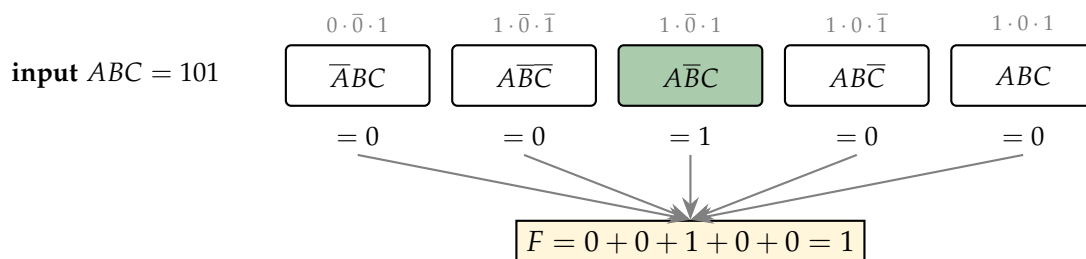
A	B	C	F	minterm if $F = 1$
0	0	0	0	
0	0	1	0	
0	1	0	0	
0	1	1	1	$\overline{A}BC$
1	0	0	1	$A\overline{B}\overline{C}$
1	0	1	1	$A\overline{B}C$
1	1	0	1	$AB\overline{C}$
1	1	1	1	ABC

$$\Rightarrow F = \overline{A}BC + A\overline{B}\overline{C} + A\overline{B}C + AB\overline{C} + ABC$$

7.7 SOP: why does it work?

From the slides:

SOP Form: Why does it work? For input $ABC = 101$, only the shaded product term — $A\overline{B}C = 1 \cdot \overline{0} \cdot 1 = 1$ — will be 1. No other product terms will “turn on”; they will all be 0. So if inputs ABC correspond to a product term in the expression, we get $0 + 0 + \dots + 1 + \dots + 0 + 0 = 1$ for output. If inputs ABC do not correspond to any product term, we get $0 + 0 + \dots + 0 = 0$.



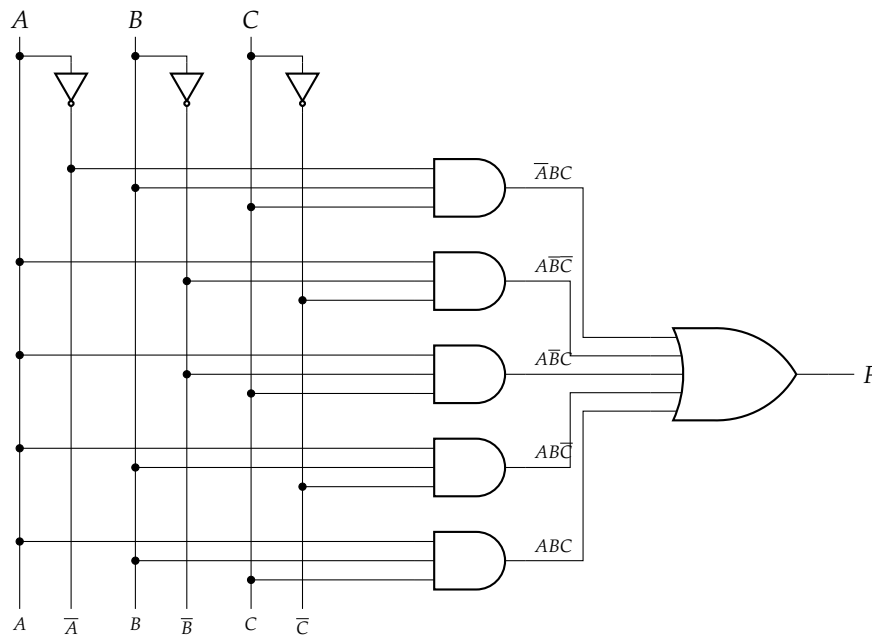
7.7.1 Proof that canonical SOP always equals F

Let S be the set of rows where $F = 1$ and $F_{\text{SOP}} = \sum_{r \in S} m_r$.

- **Each minterm is 1 in exactly its own row.** m_r has one literal per variable, chosen so that each literal is 1 precisely when that variable takes its row- r value. All literals are 1 only when *every* variable matches row r .
- **Row $r \in S$:** $m_r = 1$, so $F_{\text{SOP}} = \dots + 1 + \dots = 1 = F$ (T2).
- **Row $r \notin S$:** no minterm in the sum belongs to r , so all are 0 and $F_{\text{SOP}} = 0 = F$.

The two agree on every row, so they are the same function. Since every function has a truth table, **every Boolean function can be written in SOP form** — the slide’s claim. (The constant 0 is the empty sum.)

7.8 From SOP to a circuit: two-level logic



Level 1: one AND per minterm (inverters supply complemented inputs). Level 2: one OR. Any function becomes a circuit with at most two gate delays after the inverters.

Formula : cost of canonical SOP

For a function of n inputs with k ones in its truth table:

$$\# \text{AND gates} = k \quad [n\text{-input gates}], \quad \# \text{OR gates} = 1 \quad [k\text{-input}], \quad \# \text{inverters} \leq n.$$

For our example: 5 three-input ANDs, one five-input OR, 3 inverters. But the same function is $F = A + BC$ (Week 3 derives this): one 2-input AND and one 2-input OR. **Canonical is not minimal.**

Beyond the Slides : SOP for the tic-tac-toe function

The expression W in Chapter 3 is a sum of products but *not* canonical: its terms contain 3 of the 9 variables. The canonical SOP would have one 9-literal minterm for each of the 282 rows where $W = 1$ — 282 nine-input AND gates. The compact 8-term expression is the “minimised” form, found here by thinking rather than by algebra.

Part II

Practice

Chapter 8

Numerical Problems with Worked Solutions

Try each problem before reading its solution.

8.1 Truth tables and counting

Problem 1 *Table sizes*

How many rows does the truth table of a function with (a) 4, (b) 6, (c) 10 inputs have? How many rows for the slide's tic-tac-toe function?

Solution

$R = 2^n$: (a) 16, (b) 64, (c) 1 024. Tic-tac-toe has $n = 9$ cell bits: $2^9 = 512$ rows.

Problem 2 *How many functions?*

How many distinct Boolean functions exist of (a) 3 inputs, (b) 4 inputs?

Solution

$F(n) = 2^{2^n}$. (a) $2^8 = 256$. (b) $2^{16} = 65\,536$. Each function is one choice of the output column.

Problem 3 *Truth table storage*

A combinational circuit compares two 8-bit numbers and outputs 1 bit. If you stored its truth table's output column in a memory, how many bits would you need?

Solution

Inputs: $8 + 8 = 16$. Rows: $2^{16} = 65\,536$, one output bit each: $65\,536$ bits $= 8\,192$ bytes $= 8$ KiB. Doubling to 16-bit numbers (32 inputs) would need 2^{32} bits $= 512$ MiB — the exponential blow-up the slide warns about.

Problem 4 *Rows covered by a product term*

Over 5 variables A, B, C, D, E , in how many truth-table rows is (a) $\overline{A}BD$ equal to 1? (b) the minterm $\overline{A}B\overline{C}DE$? (c) the maxterm $A + \overline{B} + C + D + \overline{E}$ equal to 0?

Solution

(a) $k = 3$ literals: $2^{5-3} = 4$ rows (C, E free). (b) $k = 5$: $2^0 = 1$ row. (c) A maxterm is 0 only when every literal is 0: exactly 1 row ($A = 0, B = 1, C = 0, D = 0, E = 1$).

Problem 5 *Listing the rows of a product term*

For 4 variables A, B, C, D , list the rows (as $ABCD$) where $\overline{AC} = 1$.

Solution

$A = 0, C = 1$; B, D free: $2^{4-2} = 4$ rows: 0010, 0011, 0110, 0111 (decimal 2, 3, 6, 7).

Problem 6 *Tic-tac-toe circuit size*

For the 8-term expression W of Chapter 3, how many AND gates, how many AND-gate inputs in total, and how many inputs on the OR gate? How does this compare with canonical SOP (which has 282 minterms)?

Solution

8 three-input ANDs = 24 AND inputs; one 8-input OR. Canonical: 282 nine-input ANDs ($282 \times 9 = 2538$ inputs) and a 282-input OR — over 100 times larger.

Problem 7 *Tic-tac-toe evaluation*

X occupies A_{11}, A_{22}, A_{33} and A_{12} ; all other cells are O. Find W and say which term fires.

Solution

$A_{11}A_{22}A_{33} = 1 \cdot 1 \cdot 1 = 1$ (main diagonal). Every other term contains a 0: e.g. $A_{11}A_{12}A_{13} = 1 \cdot 1 \cdot 0 = 0$. So $W = 1$.

8.2 Evaluating expressions and precedence

Problem 8 *Precedence*

Evaluate $W \cdot \overline{X} + Y \cdot Z$ for (a) $W = 1, X = 0, Y = 0, Z = 1$; (b) $W = 1, X = 1, Y = 1, Z = 0$.

Solution

(a) $1 \cdot \overline{0} + 0 \cdot 1 = 1 \cdot 1 + 0 = 1$. (b) $1 \cdot \overline{1} + 1 \cdot 0 = 0 + 0 = 0$.

Problem 9 *Why brackets matter*

Find input values where $A + B\overline{C}$ and $(A + B)\overline{C}$ differ.

Solution

Try $A = 1, B = 0, C = 1$: $A + B\overline{C} = 1 + 0 = 1$; $(A + B)\overline{C} = 1 \cdot 0 = 0$. They differ, so $A + B\overline{C}$ (AND first) must not be read as $(A + B)\overline{C}$.

Problem 10 *Bar over a group*

Evaluate $\overline{A + B}$ and $\overline{A} + B$ at $A = 0, B = 1$.

Solution

$\overline{0+1} = \overline{1} = 0$, whereas $\overline{0} + 1 = 1$. A long bar is a bracket.

8.3 Duals and complements

Problem 11 *Duals*

Write the dual of (a) $A + B \cdot 0$; (b) $A(\overline{B} + C) + 1$; (c) $X\overline{Y} + \overline{X}Y$.

Solution

Swap $\cdot \leftrightarrow +$, $0 \leftrightarrow 1$, keep literals, keep grouping. (a) $A \cdot (B + 1)$. (b) $(A + \overline{B}C) \cdot 0$. (c) $(X + \overline{Y})(\overline{X} + Y)$.

Note (a): $A + B \cdot 0 = A$ and its dual $A(B + 1) = A$ — both true, as duality promises.

Problem 12 *Complement with De Morgan*

Find $\overline{F}$ for $F = (A + \overline{B})C$.

Solution

$\overline{F} = \overline{A + \overline{B} + C} = \overline{AB} + \overline{C} = \overline{A}B + \overline{C}$. Check $A = 0, B = 1, C = 1$: $F = (0 + 0) \cdot 1 = 0$, $\overline{F} = 1 \cdot 1 + 0 = 1$. ✓

Problem 13 *Complement of an SOP*

Find $\overline{F}$ for $F = AB + \overline{C}D$ and show it is a product of sums.

Solution

$\overline{F} = \overline{AB} \cdot \overline{\overline{C}D} = (\overline{A} + \overline{B})(C + \overline{D})$: a product of two sum terms. (The complement of an SOP is always a POS — the key idea behind Week 3's POS form.)

Problem 14 *Three-variable De Morgan*

Show $\overline{\overline{A} + \overline{B} + \overline{C}} = ABC$ and interpret it for a NAND gate.

Solution

De Morgan: $\overline{\overline{A} + \overline{B} + \overline{C}} = \overline{\overline{A}} \overline{\overline{B}} \overline{\overline{C}} = ABC$ (involution). So $\overline{\overline{A} + \overline{B} + \overline{C}} = \overline{ABC}$: a 3-input NAND equals an OR of the inverted inputs.

8.4 Simplification

Problem 15 *Combining*

Simplify $F = AB + A\overline{B}$.

Solution

$$F = A(B + \bar{B}) = A \cdot 1 = A \text{ (T9).}$$

Problem 16 *Redundant literal*

Simplify $F = A + \bar{A}B$.

Solution

$$F = (A + \bar{A})(A + B) = 1 \cdot (A + B) = A + B \text{ (A5, A6, A4 — T11D).}$$

Problem 17 *Dual combining*

Simplify $F = (A + B)(A + \bar{B})$.

Solution

$$F = A + B\bar{B} = A + 0 = A \text{ (A5, A6, A4 — T9D).}$$

Problem 18 *Three terms*

Simplify $F = ABC + \bar{A}\bar{B}C + AB\bar{C}$.

Solution

$ABC + \bar{A}\bar{B}C = AC(B + \bar{B}) = AC$. Then $F = AC + AB\bar{C} = A(C + B\bar{C}) = A(C + B)$ (T11D). So $F = A(B + C) = AB + AC$.

Problem 19 *Two-variable canonical SOP*

Simplify $F = \bar{A}\bar{B} + \bar{A}B + AB$.

Solution

$\bar{A}\bar{B} + \bar{A}B = \bar{A}$ (T9). $F = \bar{A} + AB = \bar{A} + B$ (T11D). Only the row $A = 1, B = 0$ gives 0, which matches $\bar{A} + B$.

Problem 20 *Consensus*

Simplify $F = XY + \bar{X}Z + YZ$.

Solution

$YZ = YZ(X + \bar{X}) = XYZ + \bar{X}YZ$. Then $F = (XY + XYZ) + (\bar{X}Z + \bar{X}YZ) = XY + \bar{X}Z$ (T10 twice). The YZ gate is redundant.

Problem 21 *Dual consensus*

Simplify $G = (A + B)(\bar{A} + C)(B + C)$.

Solution

This is the dual of Problem 20 with $X = A$, $Y = B$, $Z = C$: $G = (A + B)(\bar{A} + C)$. (Check by expanding: $(A + B)(\bar{A} + C) = AC + \bar{A}B + BC = AC + \bar{A}B$ by consensus; and G multiplies this by $(B + C)$, which it already implies.)

Problem 22 *Majority function*

$M(A, B, C) = 1$ when at least two inputs are 1. Write its canonical SOP and simplify.

Solution

Ones at 011, 101, 110, 111: $M = \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC$.

Use T3 to duplicate ABC ($ABC = ABC + ABC + ABC$) and pair it with each other term: $\bar{A}BC + ABC = BC$; $A\bar{B}C + ABC = AC$; $AB\bar{C} + ABC = AB$. So $M = AB + BC + AC$.

Problem 23 *Cost comparison*

Count gates and gate inputs for the canonical and simplified majority function (ignore inverters).

Solution

Canonical: 4 three-input ANDs (12 inputs) + one 4-input OR (4 inputs) = 5 gates, 16 inputs.
Simplified: 3 two-input ANDs (6) + one 3-input OR (3) = 4 gates, 9 inputs. And the simplified form needs no inverters at all.

8.5 Proofs

Problem 24 *A proof from axioms*

Prove $X(\bar{X} + Y) = XY$ citing axioms.

Solution

$X(\bar{X} + Y) = X\bar{X} + XY$ (A5) = $0 + XY$ (A6) = $XY + 0$ (A3) = XY (A4).

Problem 25 *Absorption, dual form*

Prove $X(X + Y) = X$.

Solution

$X(X + Y) = (X + 0)(X + Y)$ (A4) = $X + 0 \cdot Y$ (A5) = $X + 0$ (T2D) = X (A4). This is the exact dual of the slide's EX2 proof, line by line.

Problem 26 *Associativity by perfect induction*

Verify $(X + Y) + Z = X + (Y + Z)$ for all inputs.

Solution

X	Y	Z	$X + Y$	$(X + Y) + Z$	$Y + Z$	$X + (Y + Z)$
0	0	0	0	0	0	0
0	0	1	0	1	1	1
0	1	0	1	1	1	1
0	1	1	1	1	1	1
1	0	0	1	1	0	1
1	0	1	1	1	1	1
1	1	0	1	1	1	1
1	1	1	1	1	1	1

The two result columns match in all 8 rows. (Both equal 0 only when all inputs are 0.)

8.6 Canonical SOP**Problem 27** *XOR from its table*

$F(A, B)$ is 1 exactly when $A \neq B$. Write its canonical SOP.

Solution

Ones at $AB = 01$ and 10 : $F = \overline{A}B + A\overline{B}$. This is XOR, $A \oplus B$ (Weeks 3–4).

Problem 28 *From expression to table and minterms*

Build the truth table of $F = A + \overline{B}C$ and write its canonical SOP.

Solution

$F = 1$ whenever $A = 1$ (rows 100–111) or when $B = 0, C = 1$ (rows 001, 101). Ones: 001, 100, 101, 110, 111. $F = \overline{A}\overline{B}C + \overline{A}B\overline{C} + \overline{A}BC + A\overline{B}\overline{C} + ABC$ (5 minterms).

Problem 29 *Expanding to canonical form algebraically*

Expand $F = A + BC$ into canonical SOP by multiplying in missing variables.

Solution

Use $X = X(Y + \overline{Y})$ for each missing variable:

$$A = A(B + \overline{B})(C + \overline{C}) = ABC + A\overline{B}C + A\overline{B}\overline{C} + AB\overline{C}$$

$$BC = (A + \overline{A})BC = ABC + \overline{A}BC$$

Sum, removing the duplicate ABC (T3): $F = \overline{A}BC + A\overline{B}C + A\overline{B}\overline{C} + AB\overline{C} + ABC$ — exactly the slide's example. So the slide's 5-minterm function is just $A + BC$.

Problem 30 *Canonical cost for 4 variables*

A 4-input function has 6 ones in its truth table. How many AND gates, literals and OR inputs does its canonical SOP use?

Solution

6 minterms $\Rightarrow$ 6 four-input ANDs, $6 \times 4 = 24$ literals, one 6-input OR.

8.7 Gate equivalences

Problem 31 *NAND–NAND evaluation*

$Y = \text{NAND}(\text{NAND}(A, B), \text{NAND}(C, D))$. Evaluate at $A = 1, B = 1, C = 0, D = 1$ and give Y as an SOP.

Solution

$\text{NAND}(1, 1) = 0$, $\text{NAND}(0, 1) = 1$, $Y = \text{NAND}(0, 1) = 1$. Algebra: $Y = \overline{\overline{AB} \cdot \overline{CD}} = AB + CD$; at these inputs $1 + 0 = 1$. ✓

Problem 32 *NOR as AND of complements*

Show that a 3-input NOR on A, B, C is 1 only for $A = B = C = 0$, using De Morgan.

Solution

$\overline{A + B + C} = \overline{A} \overline{B} \overline{C}$. A product is 1 only if every factor is 1, i.e. $\overline{A} = \overline{B} = \overline{C} = 1$, i.e. $A = B = C = 0$. Exactly one row of 8.

Subjective Questions with Answers

Q1. Why does the tic-tac-toe example start with a “transformation” step, and what is its weakness?

Answer

Hardware only processes 0s and 1s, so each board cell must first be encoded as a bit: $A_{IJ} = 1$ if the cell holds X, else 0. This turns the board into 9 Boolean inputs. The weakness is that “0” lumps together *empty* and *O*, so the encoding can decide whether X has won but cannot decide whether O has won. A complete encoding needs 2 bits per cell (18 inputs).

Q2. Why is a natural-language rule not an adequate specification for a circuit?

Answer

English can be ambiguous or incomplete: “any diagonal” (how many?), what to do if both players have a line, what an empty cell means. A circuit must produce a definite output for every one of the 2^n input combinations, so the specification must be exact — a truth table or a Boolean expression.

Q3. Distinguish the functional and timing specifications of a logic circuit.

Answer

The functional specification states *what* each output is for each input combination (truth table / Boolean equations). The timing specification states *when*: the delay between an input changing and the output responding (propagation delay). Two circuits can share a functional spec yet differ in speed.

Q4. Define combinational and sequential logic. Give an example of each.

Answer

Combinational logic is memoryless: outputs depend only on the current inputs (e.g. “has X won?”, an adder, a comparator). Sequential logic has memory: outputs depend on current inputs and stored history (e.g. “whose turn is it?”, a counter, a combination lock).

Q5. State the three properties the slides require of a “function” and explain each.

Answer

Uniqueness: each input combination maps to exactly one output value (many inputs may share an output). Unambiguity: the same inputs always produce the same output. No memory: the output does not depend on past inputs. Together they make a combinational circuit a mathematical function.

Q6. Why does a truth table become impractical, and what is the alternative?

Answer

A table has 2^n rows, doubling with every input: 512 rows for 9 inputs, over 4 billion for 32. A Boolean expression can describe the same function compactly (tic-tac-toe: 8 product terms instead of 512 rows) and maps directly onto gates.

Q7. Explain AND, OR and NOT in logic, arithmetic and set terms.

Answer

AND: true iff both are true; multiplication on $\{0,1\}$; intersection. OR: true iff at least one is true; addition capped at 1 ($1 + 1 = 1$); union. NOT: the opposite; $1 - A$; complement (outside the region).

Q8. What are axioms, and how do they differ from theorems in Boolean algebra?

Answer

Axioms are statements assumed true without proof (two elements, closure, commutativity, identity, distributivity, complement). Theorems (idempotence, absorption, De Morgan, ...) are derived from the axioms by chains of equalities. The axioms define the algebra; the theorems are tools for manipulating expressions.

Q9. Why is the closure axiom important for circuits?

Answer

Closure says AND and OR of values in $\{0,1\}$ are again in $\{0,1\}$. For circuits, it means a gate's output is a valid logic value that can feed any other gate input — so gates compose into arbitrarily large circuits.

Q10. Which distributive law of Boolean algebra has no counterpart in ordinary arithmetic? Prove it.

Answer

$a + bc = (a + b)(a + c)$. In arithmetic, $2 + 3 \cdot 4 = 14 \neq (2 + 3)(2 + 4) = 30$. In Boolean algebra: if $a = 1$ both sides are 1; if $a = 0$ both sides equal bc . (Or check all 8 rows.)

Q11. What is the principle of duality and why does it hold?

Answer

The dual of an expression swaps $\text{AND} \leftrightarrow \text{OR}$ and $0 \leftrightarrow 1$, leaving literals alone. If an identity is true, its dual is true. Reason: every axiom comes as a dual pair, so the dual of any proof is also a valid proof, step by step.

Q12. Distinguish the dual of an expression from its complement, with an example.

Answer

For $F = A + B\bar{C}$: dual = $A(B + \bar{C})$ (swap operators only); complement = $\bar{A}(\bar{B} + C)$ (swap operators *and* complement every literal). The dual is generally a different function; the complement is always the inverted function.

Q13. State the absorption and combining theorems and explain why each saves hardware.

Answer

Absorption: $X + XY = X$ (and $X(X + Y) = X$) — the XY gate is unnecessary. Combining: $XY + X\bar{Y} = X$ — two AND gates and an OR become a wire, because a variable that appears both plain and complemented in otherwise identical terms drops out.

Q14. Prove involution $\bar{\bar{X}} = X$ from the axioms.

Answer

First, the complement is unique: if y and z both satisfy $X + y = 1$, $Xy = 0$ (and the same for z), then $y = y(X + z) = Xy + yz = Xz + yz = (X + y)z = z$. Since $\bar{X} + X = 1$ and $\bar{X}X = 0$, X is a complement of $\bar{X}$; by uniqueness it is *the* complement: $\bar{\bar{X}} = X$.

Q15. State De Morgan's laws and interpret them in English and in gates.

Answer

$\overline{X + Y} = \bar{X}\bar{Y}$ and $\overline{XY} = \bar{X} + \bar{Y}$ (and n -variable versions). English: "not (at least one true)" = "all false"; "not (all true)" = "at least one false". Gates: NOR = AND with inverted inputs; NAND = OR with inverted inputs.

Q16. Explain bubble pushing and show that a NAND–NAND circuit computes a sum of products.

Answer

By De Morgan, a bubble can be moved from a gate's output to all its inputs if AND and OR are swapped; two bubbles on one wire cancel. In $Y = \text{NAND}(\text{NAND}(A, B), \text{NAND}(C, D))$, redraw the last NAND as an OR with bubbled inputs; those bubbles cancel the first-level NAND bubbles, leaving $Y = AB + CD$.

Q17. Define literal, product term (implicant), minterm and maxterm.

Answer

Literal: a variable or its complement. Product term / implicant (slide's sense): an AND of literals. Minterm: a product containing every input variable exactly once (plain or complemented) — 1 in exactly one row. Maxterm: a sum containing every input variable exactly once — 0 in exactly one row.

Q18. What is a canonical form, and why do we want one if the function is the same anyway?

Answer

A canonical form is a standard rule producing exactly one expression per function, giving a unique algebraic signature (like the truth table, but algebraic). Many expressions describe the same function, and different expressions lead to different gate realisations with different cost and speed; the canonical form is a reference point from which to compare and simplify.

Q19. Explain why the canonical SOP of a truth table always equals the function.

Answer

Each minterm is 1 in exactly its own row. For a row where $F = 1$, its minterm is in the sum, so the OR outputs 1. For a row where $F = 0$, no minterm in the sum belongs to that row, so every term is 0 and the OR outputs 0. The expressions agree on all rows, hence are equal.

Q20. Is the canonical SOP the best circuit? Discuss with the slide's example.

Answer

No. The slide's function $F = \overline{A}BC + A\overline{B}C + A\overline{B}\overline{C} + AB\overline{C} + ABC$ needs 5 three-input ANDs and a 5-input OR, but it simplifies to $A + BC$: one 2-input AND and one 2-input OR. Canonical SOP is unique and always correct, but usually far from minimal; minimisation is the subject of Week 3.

Chapter 10

Multiple-Choice Questions

Attempt all 40; answers and explanations follow in Section 10.2.

10.1 Questions

Specifications and truth tables

1. In the tic-tac-toe encoding, $A_{IJ} = 0$ means:
 - (a) the cell is empty
 - (b) the cell holds O
 - (c) the cell does not hold X
 - (d) the game is drawn
2. How many rows does the tic-tac-toe truth table have?
 - (a) 9
 - (b) 81
 - (c) 512
 - (d) 19 683
3. The timing specification of a circuit describes:
 - (a) the output for each input combination
 - (b) the delay between inputs changing and outputs responding
 - (c) the clock frequency only
 - (d) the number of gates
4. A combinational circuit's outputs depend on:
 - (a) current inputs only
 - (b) past inputs only
 - (c) current inputs and stored state
 - (d) the clock
5. Which is *not* a property of a function as defined on the slides?
 - (a) uniqueness
 - (b) unambiguity
 - (c) no memory
 - (d) invertibility
6. The number of distinct Boolean functions of 2 inputs is:
 - (a) 4

- (b) 8
- (c) 16
- (d) 256

7. Over 6 variables, a product of 2 literals is 1 in how many rows?

- (a) 2
- (b) 4
- (c) 16
- (d) 32

8. A minterm of n variables is 1 in:

- (a) exactly one row
- (b) exactly n rows
- (c) 2^{n-1} rows
- (d) all but one row

Operations and precedence

9. In Boolean algebra, $1 + 1 =$

- (a) 0
- (b) 1
- (c) 2
- (d) 10

10. $W\bar{X} + YZ$ is evaluated as:

- (a) $W(\bar{X} + Y)Z$
- (b) $(W\bar{X}) + (YZ)$
- (c) $\overline{W\bar{X}} + YZ$
- (d) $W(\overline{X + Y})Z$

11. The highest-precedence Boolean operator is:

- (a) OR
- (b) AND
- (c) NOT
- (d) all are equal

12. For $A = 1, B = 0, C = 1$, $(A + B)\bar{C}$ equals:

- (a) 0
- (b) 1
- (c) 2
- (d) undefined

13. In Venn-diagram terms, AND corresponds to:

- (a) union

- (b) intersection
- (c) complement
- (d) difference

Axioms, duality, theorems

14. Which distributive law is special to Boolean algebra?

- (a) $a(b + c) = ab + ac$
- (b) $a + bc = (a + b)(a + c)$
- (c) $a(bc) = (ab)c$
- (d) $a + b = b + a$

15. The closure axiom guarantees that:

- (a) $0 \neq 1$
- (b) AND/OR results stay in $\{0, 1\}$
- (c) order doesn't matter
- (d) every element has a complement

16. The dual of $A + 0$ is:

- (a) $A \cdot 1$
- (b) $\overline{A} + 1$
- (c) $A \cdot 0$
- (d) $\overline{A} \cdot 1$

17. The dual of $X(\overline{Y} + Z)$ is:

- (a) $\overline{X} + Y\overline{Z}$
- (b) $X + \overline{Y}Z$
- (c) $\overline{X}(Y + \overline{Z})$
- (d) $X(\overline{Y} + Z)$

18. $X + XY$ simplifies to:

- (a) XY
- (b) X
- (c) Y
- (d) $X + Y$

19. $XY + X\overline{Y}$ simplifies to:

- (a) X
- (b) Y
- (c) 1
- (d) $X \oplus Y$

20. $X\overline{Y} + Y$ simplifies to:

- (a) X

- (b) XY
- (c) $X + Y$
- (d) Y

21. $(X + \overline{Y})Y$ equals:

- (a) X
- (b) XY
- (c) $X + Y$
- (d) 0

22. $X \cdot X$ equals (idempotence):

- (a) X^2
- (b) X
- (c) 1
- (d) 0

23. $X + 1$ equals:

- (a) X
- (b) 1
- (c) 0
- (d) $\overline{X}$

24. In the slide proof of $X + XY = X$, the step $X(1 + Y) = X \cdot 1$ uses:

- (a) the complement axiom
- (b) the null theorem $Y + 1 = 1$
- (c) idempotence
- (d) De Morgan

25. Which statement about Boolean algebra is FALSE?

- (a) $XY = XZ$ implies $Y = Z$
- (b) $\overline{\overline{X}} = X$
- (c) $X + \overline{X} = 1$
- (d) $X \cdot 0 = 0$

26. $XY + \overline{X}Z + YZ$ simplifies to:

- (a) $XY + YZ$
- (b) $XY + \overline{X}Z$
- (c) $Y + Z$
- (d) $X + Z$

De Morgan and gates

27. $\overline{A + B + C}$ equals:

- (a) $\overline{A} + \overline{B} + \overline{C}$

- (b) $\overline{A} \overline{B} \overline{C}$
- (c) ABC
- (d) $\overline{ABC}$

28. $\overline{AB}$ equals:

- (a) $\overline{A} \overline{B}$
- (b) $\overline{A} + \overline{B}$
- (c) $A + B$
- (d) $\overline{A + B}$

29. A NOR gate is equivalent to:

- (a) OR with inverted inputs
- (b) AND with inverted inputs
- (c) NAND with inverted output
- (d) XOR

30. The complement of $AB + \overline{C}$ is:

- (a) $(\overline{A} + \overline{B})C$
- (b) $\overline{A} \overline{B} + C$
- (c) $\overline{ABC}$
- (d) $(A + B)\overline{C}$

31. Two inverters in series act as:

- (a) a NOT gate
- (b) a wire (buffer)
- (c) a NAND
- (d) an open circuit

32. $\text{NAND}(\text{NAND}(A, B), \text{NAND}(C, D))$ equals:

- (a) $(A + B)(C + D)$
- (b) $AB + CD$
- (c) $\overline{AB + CD}$
- (d) $ABCD$

33. "It is not the case that A, B, C are all false" is:

- (a) $\overline{A} \overline{B} \overline{C}$
- (b) $A + B + C$
- (c) ABC
- (d) $\overline{A + B + C}$

Definitions and SOP

34. Which is a minterm of A, B, C?

- (a) $A\overline{B}$

- (b) $A + \overline{B} + C$
- (c) $\overline{A}\overline{B}\overline{C}$
- (d) $\overline{A}$

35. Which is a maxterm of A, B, C ?

- (a) $A\overline{B}C$
- (b) $A + \overline{B} + C$
- (c) $A + B$
- (d) $\overline{A}\overline{B}\overline{C}$

36. SOP is also called:

- (a) conjunctive normal form
- (b) disjunctive normal form
- (c) maxterm expansion
- (d) product of sums

37. For the row $A = 1, B = 0, C = 1$, the minterm is:

- (a) $A\overline{B}C$
- (b) $\overline{A}\overline{B}\overline{C}$
- (c) $A + \overline{B} + C$
- (d) $\overline{A} + B + \overline{C}$

38. The canonical SOP of the slide example (ones at rows 3–7) minimises to:

- (a) $A + BC$
- (b) $AB + C$
- (c) $A + B + C$
- (d) ABC

39. A 3-input function with 4 ones has a canonical SOP with how many AND gates?

- (a) 3
- (b) 4
- (c) 8
- (d) 12

40. Why is canonical SOP guaranteed to reproduce the truth table?

- (a) each minterm is 1 in exactly its own row
- (b) it uses only NAND gates
- (c) it is always minimal
- (d) it ignores rows with $F = 0$ incorrectly

10.2 Answers and Explanations

1 c	2 c	3 b	4 a	5 d	6 c	7 c	8 a	9 b	10 b
11 c	12 a	13 b	14 b	15 b	16 a	17 b	18 b	19 a	20 c
21 b	22 b	23 b	24 b	25 a	26 b	27 b	28 b	29 b	30 a
31 b	32 b	33 b	34 c	35 b	36 b	37 a	38 a	39 b	40 a

1. **(c)** 0 means “not X” — empty and O are indistinguishable.
2. **(c)** $2^9 = 512$ ($19\,683 = 3^9$ would be the 3-state count).
3. **(b)** Timing = delay between input change and output response.
4. **(a)** Combinational = memoryless.
5. **(d)** Many inputs may map to the same output, so a function need not be invertible.
6. **(c)** $2^{2^2} = 16$.
7. **(c)** $2^{6-2} = 16$.
8. **(a)** All n variables are fixed, so exactly one row.
9. **(b)** OR saturates: $1 + 1 = 1$.
10. **(b)** NOT, then AND, then OR.
11. **(c)** NOT binds tightest.
12. **(a)** $(1 + 0) \cdot \bar{1} = 1 \cdot 0 = 0$.
13. **(b)** AND is intersection.
14. **(b)** OR distributing over AND fails for ordinary numbers.
15. **(b)** Closure: results stay in the set.
16. **(a)** Swap $+ \rightarrow \cdot$ and $0 \rightarrow 1$: $A \cdot 1$.
17. **(b)** Swap operators, keep literals: $X + \bar{Y}Z$.
18. **(b)** Absorption (T10).
19. **(a)** Combining (T9).
20. **(c)** T11D.
21. **(b)** T11: $XY + \bar{Y}Y = XY$.
22. **(b)** Idempotence; there are no powers in Boolean algebra.
23. **(b)** Null theorem.
24. **(b)** $1 + Y = 1$ is the null theorem (the slide labels it “Identity (2)”, i.e. theorem 2).
25. **(a)** No cancellation: $X = 0$ makes $XY = XZ$ for any Y, Z .
26. **(b)** Consensus theorem: YZ is redundant.
27. **(b)** De Morgan: break the bar, change $+$ to $\cdot$.
28. **(b)** De Morgan dual form.
29. **(b)** $\overline{X + Y} = \bar{X} \bar{Y}$.
30. **(a)** $\overline{AB + \bar{C}} = \overline{AB} \cdot C = (\bar{A} + \bar{B})C$.

31. **(b)** Involution: $\overline{\overline{X}} = X$.
32. **(b)** Bubble pushing turns NAND–NAND into AND–OR.
33. **(b)** “Not all false” = “at least one true” = $A + B + C$.
34. **(c)** Contains all three variables as a product.
35. **(b)** Contains all three variables as a sum.
36. **(b)** SOP = disjunctive normal form = minterm expansion.
37. **(a)** 1 plain, 0 complemented: $A\overline{B}C$.
38. **(a)** Shown in Problem “Expanding to canonical form algebraically”.
39. **(b)** One AND per 1 in the table.
40. **(a)** A row’s minterm is its unique detector.

Appendix A

Formula Sheet, Glossary and Revision

A.1 Formula and law sheet

Rows in a truth table	2^n
Number of n -input functions	2^{2^n}
Rows where a k -literal product is 1	2^{n-k} (minterm: 1 row; maxterm is 0 in 1 row)
Canonical SOP cost	k ANDs of n inputs, one k -input OR (k = number of 1s)
Identity / Null	$X + 0 = X, X \cdot 1 = X; X + 1 = 1, X \cdot 0 = 0$
Idempotent / Involution	$X + X = X, XX = X; \overline{\overline{X}} = X$
Complement	$X + \overline{X} = 1, X\overline{X} = 0$
Commutative / Associative	$X + Y = Y + X \dots; (X + Y) + Z = X + (Y + Z) \dots$
Distributive	$X(Y + Z) = XY + XZ; X + YZ = (X + Y)(X + Z)$
Combining	$XY + X\overline{Y} = X; (X + Y)(X + \overline{Y}) = X$
Absorption	$X + XY = X; X(X + Y) = X$
Redundant literal	$X\overline{Y} + Y = X + Y; (X + \overline{Y})Y = XY$
Consensus (beyond)	$XY + \overline{X}Z + YZ = XY + \overline{X}Z$
De Morgan	$\overline{X_1 + \dots + X_n} = \overline{X_1} \dots \overline{X_n}; \overline{\overline{X_1} \dots \overline{X_n}} = \overline{X_1} + \dots + \overline{X_n}$
Duality	swap $\cdot \leftrightarrow +, 0 \leftrightarrow 1$; keep literals
Complement of expression	dual + complement every literal
Expand to canonical	multiply a term missing Y by $(Y + \overline{Y})$
Precedence	NOT > AND > OR

A.2 Glossary

Axiom A statement assumed true; the starting point of the algebra.

Bubble Small circle on a gate pin denoting inversion.

Canonical form A standard expression that is unique for each function.

Combinational logic Memoryless logic; outputs depend only on current inputs.

Disjunctive normal form (DNF) Another name for sum of products.

Dual Expression with AND/OR and 0/1 swapped.

Functional specification Relationship between input and output variables.

Implicant Product (AND) of literals (textbooks: one that implies F).

Literal A variable or its complement.

Maxterm Sum including every variable once; 0 in exactly one row.

Minterm Product including every variable once; 1 in exactly one row.

Perfect induction Proof by checking every input combination.

Sequential logic Logic with memory; outputs depend on history too.

SOP (sum of products) OR of AND terms; canonical SOP = OR of the minterms where $F = 1$.

Timing specification Delay between input change and output response.

Truth table Exhaustive list of outputs for all 2^n inputs.

A.3 Revision checklist

- ☐ Encode tic-tac-toe as bits; explain why O-wins needs more bits.
- ☐ Functional vs timing spec; combinational vs sequential.
- ☐ Three properties of a function.
- ☐ 2^n rows, 2^{2^n} functions, 2^{n-k} rows per product term.
- ☐ Write W for tic-tac-toe (8 terms) and draw its circuit.
- ☐ NOT/AND/OR truth tables, arithmetic and Venn readings.
- ☐ Six axioms; the “unusual” distributive law and its proof.
- ☐ Duality: form duals; why it works; dual $\neq$ complement.
- ☐ Theorems T1–T12 with duals; prove T2, T3, T9, T10, T11 from axioms.
- ☐ De Morgan: statement, proof, English, NAND/NOR equivalences, bubble pushing.
- ☐ Literal, implicant, minterm, maxterm; precedence.
- ☐ Canonical SOP recipe and proof of correctness; canonical $\neq$ minimal.